QUICKBILL

Billing and Invoice Management System for MSMEs

Academic Case Study Report

Project Type: Academic Software Engineering Project

Technology Stack: Python, Flask, MySQL, HTML, CSS, JavaScript, Pandas, OpenPyXL, ReportLab, Docker and Gunicorn

1. Executive Summary

QuickBill is a web-based billing and invoice management system designed for small and medium-sized businesses. The application centralizes customer management, product and service management, invoice creation, staff administration and business reporting in a role-based environment. The implementation uses Flask for the web layer and MySQL for persistent data storage. Docker and Docker Compose are included to provide a repeatable application and database deployment environment.

The system is organized around three operational roles: ADMIN, MANAGER and STAFF. Administrative users manage the business tenant, employees, customers, products and reports. Managers can work with products, customers and billing activities, while staff members can perform operational billing tasks. The codebase also supports importing spreadsheet data into the billing workflow and generating invoice documents.

2. Problem Statement

Many small businesses manage invoices, customer records and sales information using disconnected spreadsheets or manual documents. Such processes can result in duplicate data entry, inconsistent invoice calculations, difficulty tracking customer history and limited visibility into business performance. A centralized application is therefore required to provide controlled access to business data and streamline recurring billing operations.

3. Objectives

- Develop a centralized web application for billing and invoice management.
- Provide role-based access for ADMIN, MANAGER and STAFF users.
- Maintain customers, products and services in a relational database.
- Create invoices containing multiple line items with subtotal, tax and total values.
- Support Excel-based data input using Pandas and OpenPyXL-compatible spreadsheet handling.
- Provide business analytics covering revenue, invoices, staff activity, customers and products.
- Isolate business data using tenant identifiers so records are associated with the appropriate business.
- Package the application and MySQL database with Docker Compose for consistent deployment.

4. Scope of the System

The implemented scope covers authentication, tenant creation, employee management, customer management, product and service management, invoice creation, invoice viewing, spreadsheet-assisted billing, business settings and administrative reporting. The system is intended as an academic prototype and can be extended with production-grade authentication, audit logging, payment integration, inventory automation and stronger authorization controls.

5. Functional Requirements

Module	Implemented Capability	Primary Role
Authentication	Login, logout and password hashing using SHA-256	All users
Tenant Management	Create a business tenant during	ADMIN

	administrator registration	
User Management	Add, delete and update employee roles	ADMIN
Customer Management	Create and view customer records and invoice history	ADMIN / MANAGER / STAFF
Product Management	Add and view products and services	ADMIN / MANAGER
Invoice Management	Create multi-item invoices and view generated invoices	MANAGER / STAFF
Excel Integration	Read spreadsheet rows for customer/product and invoice processing	MANAGER / STAFF
Reporting	Revenue, invoice count, staff activity, top customers and top products	ADMIN
Business Settings	Maintain business name, GST number, address, tax and currency settings	ADMIN

6. Non-Functional Requirements

- Usability: provide simple browser-based forms and dashboards for common billing tasks.
- Maintainability: separate Flask routes, templates, static styling and database schema.
- Security: restrict routes according to the active session role and associate records with a tenant identifier.
- Portability: provide Docker and Docker Compose configuration for the Flask service and MySQL service.
- Performance: load reusable application components and execute focused SQL queries for dashboards and reports.
- Extensibility: keep business entities such as tenants, users, customers, products, invoices and invoice items as separate relational tables.

7. System Architecture

System Flow

[Browser / User Interface]
↓
[Flask Web Application]
↓
[Authentication + Role Authorization]
↓
[Business Logic + Invoice Processing + Excel Processing + Reporting]
↓
[MySQL Relational Database]
↓
[Docker Volume for Persistent Database Data]

The web application is served by Flask and can be executed through Gunicorn inside the supplied Docker image. The Docker Compose configuration defines a MySQL 8.0 database service and a web service. The database is persisted through a named Docker volume, while the initialization SQL script creates the QuickBill schema and sample records.

8. Database Design

Entity	Important Attributes	Purpose
tenants	id, business_name, email	Represents a business and provides tenant-level isolation.
users	id, tenant_id, username, password, role	Stores application users and their access roles.
customers	id, tenant_id, name, email, phone	Stores customer information for each business.
products	id, tenant_id, name, type, price, stock	Stores products and services.
invoices	id, tenant_id, customer_id, invoice_number, dates, subtotal, tax, total, status	Stores invoice-level transaction data.

invoice_items	invoice_id, item_name, unit_price, quantity, line_total	Stores individual items belonging to an invoice.

The database schema uses primary keys for entity identification and foreign keys for relationships. Invoice items reference invoices with cascading deletion in the supplied schema. Tenant identifiers are included in the main business entities so queries can restrict records to the active business.

9. User Roles and Access Control

Role	Responsibilities
ADMIN	Creates the business tenant, manages employees, views customers and products, accesses reports and changes business settings.
MANAGER	Works with products and customers and participates in invoice creation and operational billing.
STAFF	Performs invoice creation and operational billing activities and can access the staff dashboard.

Route-level checks are implemented in Flask using the session role. Unauthorized roles receive an access-denied response or are redirected to the appropriate dashboard. The application also stores the tenant identifier in the session and uses it in database queries.

10. Invoice Processing Workflow

1. The user signs in and the application establishes the role and tenant context in the session.
2. The billing user opens the invoice creation page and selects a customer and products, or supplies spreadsheet data.
3. For manual billing, item data, subtotal, tax and total values are collected from the form.
4. For Excel-based billing, Pandas reads the uploaded .xlsx file and calculates line totals, subtotal, tax and total.
5. The main invoice record is inserted into the invoices table.
6. Each invoice line is inserted into invoice_items with item name, unit price, quantity and line total.
7. The transaction is committed and the application redirects to the invoice view.

11. Excel Integration

QuickBill contains spreadsheet-assisted workflows using Pandas. The invoice creation route can read columns such as product_name, price, quantity and customer_name from an uploaded Excel file. A separate extraction endpoint reads customer_name, product_name and price values and inserts missing customer and product records for the active tenant. This reduces repetitive manual entry when existing business data is already maintained in spreadsheets.

12. Reporting and Analytics

Report Area	Data Derived
Revenue Summary	Sum of invoice totals and total invoice count for the active tenant.
Staff Performance	Invoice activity and sales totals associated with users in the tenant.
Top Customers	Customer invoice counts and revenue, limited to the highest-revenue records.
Top Products	Units sold and revenue calculated from invoice item records.
Product View	Product type, price, units sold and revenue for administrative analysis.

13. Implementation Details

The application entry point is app.py. Flask routes map browser requests to authentication, dashboards, CRUD operations, invoice processing and reporting functions. mysql-connector-python is used to establish database connections. Passwords entered during registration and login are transformed using SHA-256 before

comparison/storage. Pandas is used for spreadsheet processing, while ReportLab and python-docx are included in the dependency set for document generation capabilities.

The supplied Dockerfile uses Python 3.10, installs the requirements and launches the Flask application through Gunicorn on port 5000. Docker Compose exposes the web service on host port 5001 and MySQL on host port 3307, while the web container connects to the database service through environment variables.

14. Testing Strategy

Test Category	Example Test	Expected Result
Authentication	Valid and invalid login attempts	Valid users reach their role dashboard; invalid credentials are rejected.
Authorization	Access an ADMIN route using another role	Request is denied or redirected.
Customer Management	Create and retrieve a customer within a tenant	Customer is stored and displayed for that tenant.
Product Management	Add a product with name, type and price	Product is inserted and available to billing workflows.
Invoice Creation	Create invoice with multiple items	Invoice and invoice_items records are stored and totals are displayed.
Excel Processing	Upload a valid .xlsx file	Rows are read and billing/customer/product processing occurs.
Reporting	Open reports as ADMIN	Revenue, invoice, staff, customer and product summaries are displayed.
Containerization	Start services using Docker Compose	Web and MySQL services start with the configured ports and volume.

15. Results and Observations

The completed implementation demonstrates a functional end-to-end billing workflow from authentication and customer selection through invoice persistence and reporting. The separation of role-specific dashboards provides a clear operational structure. The relational schema supports invoice-level and item-level data, while tenant identifiers provide a basis for multi-business data separation.

The project also demonstrates practical integration between a Python web application, relational database, spreadsheet processing and containerized deployment. The implementation can therefore serve as an academic demonstration of software engineering concepts including modular design, database normalization, role-based authorization and deployment packaging.

16. Limitations

- The supplied application uses a development-oriented secret-key fallback and database credentials in the Docker Compose configuration; production deployment should use secure secret management.
- SHA-256 password hashing is implemented, but a password-specific adaptive hash such as Argon2 or bcrypt would be preferable for production authentication.
- Some database and route logic is prototype-oriented and would benefit from stronger validation, transactions and centralized error handling.
- The supplied reporting queries should be reviewed for business semantics before being used for audited financial reporting.
- The system does not include a payment gateway, automated reminders, tax-compliance integration or comprehensive audit logs.

17. Future Enhancements

- Implement Argon2 or bcrypt password hashing and stronger session security.
- Add granular permission policies and audit trails for sensitive operations.

- Integrate payment gateways and automated invoice delivery.
- Add inventory movement, low-stock alerts and purchase management.
- Provide exportable financial reports and configurable tax rules.
- Add automated backups, health checks, structured logging and production monitoring.
- Introduce automated unit and integration tests in the deployment pipeline.

18. Conclusion

QuickBill demonstrates how a small-business billing workflow can be consolidated into a role-based web application. The project combines Flask application development, relational database design, spreadsheet integration, reporting and Docker-based deployment. Its architecture provides a practical foundation that can be strengthened with production security, testing and operational controls.

19. References and Project Sources

1. Flask documentation:

https://flask.palletsprojects.com/en/stable/

2. MySQL documentation:

https://dev.mysql.com/doc/

3. Pandas documentation:

https://pandas.pydata.org/docs/

4. Docker documentation:

https://docs.docker.com/

5. Gunicorn documentation:

https://docs.gunicorn.org/

6. Python documentation:

https://docs.python.org/3/